MACHINE LEARNING APPLIQUÉ · 3e année DSI

ATELIER 3 · 3 heures

La voix et l'image

Brancher un modèle audio et un modèle de vision — version ÉTUDIANT

Mission Le guichet de l'atelier 2 reçoit maintenant des messages vocaux laissés sur le répondeur et des captures d'écran envoyées par l'application. Votre classifieur ne lit que du texte. Vous allez brancher un modèle qui écoute, puis un modèle qui voit, et mesurer ce que chaque canal fait perdre — avec la même méthode qu'à l'atelier 2.

1. Objectifs

- Transcrire un message vocal avec Whisper.
- Mesurer la qualité de la transcription avec le WER.
- Utiliser le texte transcrit avec le classifieur de l'atelier 2.
- Classer une capture d'écran avec un modèle de vision.
- Comparer les résultats des canaux texte, voix et image.

2. Deux façons d'intégrer un modèle

Un modèle audio ne renvoie pas une catégorie : il renvoie du texte. Pour la voix, on construit donc une chaîne de deux modèles. Pour l'image, un seul modèle suffit : il regarde la capture et répond directement. On dit qu'il travaille de bout en bout (end-to-end).

```
PARTIE 1 — une chaîne de deux modèles

 message    +-------------+   texte    +--------------+   catégorie
 vocal ---> |   Whisper   | ---------> | classifieur  | ----------->
 (.mp3)     |    (ASR)    |            | (atelier 2)  |  facturation
            +-------------+            +--------------+
             mesuré par le WER          mesuré par le macro-F1


PARTIE 2 — un seul modèle, de bout en bout

  capture    +------------------------------+     catégorie
  d'écran -> |  modèle de vision (qwen3.8)  | --> compte
  (.png)     |  il lit l'image ET la consigne|
             +------------------------------+
              mesuré par le macro-F1
```

ASR veut ***dire Automatic Speech Recognition*** : la transcription automatique de la parole. Whisper est le modèle ASR le plus utilisé ; Groq le sert gratuitement.

La règle d'une chaîne Les erreurs se propagent : si Whisper entend « fracture » au lieu de « facture », le classifieur ne peut plus rien rattraper. On mesure donc CHAQUE étage séparément : le WER pour Whisper, le macro-F1 pour la chaîne entière. Sans cela, on ne sait pas quel étage réparer.

Le WER : compter les mots mal entendus

Le WER (Word Error Rate) aligne la transcription sur le texte de référence, mot à mot, et compte trois sortes d'erreurs.

```
référence     : ma facture de mars est de 89 euros
transcription : ma facture de mars  et de 89 euros merci
                                    S                I
  S  substitution : un mot remplacé par un autre    (est -> et)
  D  deletion     : un mot oublié
  I  insertion    : un mot en trop                  (merci)
  WER = (S + D + I) / N  =  (1 + 0 + 1) / 8  =  0,25
  N = nombre de mots de la RÉFÉRENCE ; 0 = transcription parfaite
```

Avant de compter, on normalise les deux textes : minuscules, sans ponctuation, « € » écrit « euros ». Sinon « Euros. » et « euros » comptent comme une erreur alors que Whisper a parfaitement entendu. Sur la phrase d'exemple du fichier wer.py, le WER passe de 0,545 sans normalisation à 0,083 avec : sur six erreurs comptées, une seule était une erreur d'écoute.

3. Les données

Fichier	Contenu
data/messages.csv	72 lignes : id, fichier, voix, texte_oral, categorie. Ce sont les 72 tickets de l'atelier 2
data/voix/T001.mp3 …	Chaque ticket lu par une voix de synthèse (3 voix : mb-fr1, mb-fr4, fr), en qualité téléphone
data/images.csv, data/captures/	20 captures d'écran de l'application fictive Fibrelune, 5 par catégorie

La colonne texte_oral est le ticket tel qu'on le prononce : accents rétablis, « jai » devenu « j'ai ». C'est le texte lu par la voix de synthèse, celui que vous lirez, et la référence du WER. Les catégories sont celles de l'atelier 2 : même règle d'annotation, mêmes 72 tickets — seul le canal change.

4. Préparation

Travail en binôme. Utilisez la clé Groq de l'atelier 2. Vérifiez que le dossier contient les données audio, les captures d'écran et le classifieur. La variable GROQ_API_KEY doit être configurée. Ne partagez jamais votre clé API.

```
cd ~/tp3-devops
source .venv/bin/activate
cd code
python -m pip install groq jiwer mutagen
```

5. Travail pratique

PARTIE 1 — LA VOIX

Étape 1 – Transcrire un message vocal

Dans transcription.py, complétez transcrire() à partir du modèle fourni. Le fichier audio doit être ouvert en mode binaire (« rb »). Exécutez les commandes:

```
python transcription.py ../data/voix/T016.mp3
python transcription.py ../data/voix/T006.mp3 whisper-large-v3-turbo auto
```

Comparez la transcription obtenue avec le texte de référence. Que permet le paramètre language="fr" ?

Étape 2 – Calculer le WER

Complétez uniquement normaliser() dans wer.py : minuscules, conversion de € en « euros », suppression de la ponctuation et réduction des espaces multiples. Conservez les accents.

Ecrivez le fichier test_wer.py pour tester le bon fonctionnement de la fonction normaliser(); et Exécutez :

```
python test_wer.py          # 8/8 tests reussis
python wer.py               # brut 0.545, normalise 0.083, et l'alignement
```

Rappel : WER = (S + D + I) / N. N représente le nombre de mots de la référence. Un WER de 0 signifie que la transcription est parfaite.

Étape 3 – Évaluer la chaîne vocale

Cette étape permet de comparer les performances du classifieur lorsqu'il reçoit le texte original puis le texte transcrit automatiquement par Whisper. Exécutez :

```
python evaluer_voix.py texte                    # ~4 min
python evaluer_voix.py whisper-large-v3-turbo   # ~9 min la 1re fois
```

Comparez le WER, le macro-F1 et la latence pour observer l'impact de la transcription vocale sur la qualité de classification et le temps de traitement.

	WER	macro-F1	Latence / message
Texte lu (référence)	—		
Voix -> whisper-large-v3-turbo			
Écart (en tickets)			

Étape 4 – Observer la propagation des erreurs

Cette étape consiste à identifier les tickets pour lesquels la transcription vocale a entraîné une mauvaise classification, alors que le texte original était correctement classé.

Complétez propagation() en comparant la vraie catégorie, la prédiction sur le texte et la prédiction après transcription.

```
python test_propagation.py
python propagation.py whisper-large-v3-turbo
```

Examinez deux tickets perdus. Quel mot a été mal transcrit ? Comment cette erreur a-t-elle influencé la catégorie ?

PARTIE 2 — L'IMAGE

Étape 5 – Classer une capture d'écran

Cette étape consiste à utiliser un modèle de vision pour analyser directement une capture d'écran et prédire la catégorie du ticket (facturation, compte, etc.).

Dans *vision.py*, complétez *classer_image()* en reprenant le principe de *classer()* de l'atelier 2. Le message envoyé au modèle contient la consigne et l'image encodée par *data_url(chemin)*.

Après avoir testé une image avec *vision.py*, lancez l'évaluation sur les 20 captures pour calculer le macro-F1 et examinez une image mal classée afin de comprendre pourquoi le modèle s'est trompé.

Exécutez :

```
python vision.py ../data/captures/I06.png
python evaluer_images.py
```

Étape 6 – Comparer les trois canaux

Exécutez python comparer.py, puis complétez le tableau.

Canal	Modèle	Qualité	Latence	Coût indicatif
Texte	Classifieur atelier 2	Macro-F1 : ____	____	____
Voix	Whisper + classifieur	WER : ____ / Macro-F1 : ____	____	____
Image	Modèle de vision	Macro-F1 : ____	____	____

6. Questions de conclusion

A. Quel canal donne le meilleur résultat ? Justifiez avec une mesure.
B. Pour la voix, les erreurs viennent-elles de la transcription ou de la classification ?
C. Quelles vérifications faudrait-il faire avant d'utiliser ce système dans un vrai guichet ?

7. Livrable

A. Les fichiers complétés : transcription.py, wer.py, propagation.py et vision.py.
B. Les résultats des évaluations.
C. Le tableau comparatif des trois canaux.
D. Les réponses aux questions de conclusion.

8. À retenir

Pour la voix, deux modèles interviennent : Whisper transforme la parole en texte, puis le classifieur prédit la catégorie. Une erreur de transcription peut donc modifier la classification. Pour l'image, le modèle de vision analyse directement la capture. La comparaison doit tenir compte de la qualité, de la latence et du coût.